



NAME : ANNEPU MAHARSHI

REGNO: 20233059

SEC : A2

NAME : ANNFPJ MAHARSHI

REGNO: 20233059

SEC : A2

```

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, random_split
import matplotlib.pyplot as plt
import numpy as np

transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])

dataset = torchvision.datasets.MNIST(root="./data", train=True, transform=transform, download=True)
test_dataset = torchvision.datasets.MNIST(root="./data", train=False, transform=transform, download=True)

train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = random_split(dataset, [train_size, val_size])

batch_size = 64
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

class ANN(nn.Module):
    def __init__(self, input_size=784, hidden1=128, hidden2=64, output_size=10, dropout_rate=0.5, l2_lambda=0.001):
        super(ANN, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden1)
        self.fc2 = nn.Linear(hidden1, hidden2)
        self.fc3 = nn.Linear(hidden2, output_size)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout_rate)
        self.l2_lambda = l2_lambda

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.relu(self.fc2(x))
        x = self.dropout(x)
        x = self.fc3(x)
        return x

model = ANN()

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=model.l2_lambda)

def train(model, train_loader, val_loader, criterion, optimizer, epochs=10):
    train_losses, val_losses = [], []
    for epoch in range(epochs):
        model.train()
        train_loss = 0
        for images, labels in train_loader:
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            train_loss += loss.item()

        model.eval()
        val_loss = 0
        with torch.no_grad():
            for images, labels in val_loader:
                outputs = model(images)
                loss = criterion(outputs, labels)
                val_loss += loss.item()

    return train_losses, val_losses

```

```

        for images, labels in val_loader:
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

    train_loss /= len(train_loader)
    val_loss /= len(val_loader)
    train_losses.append(train_loss)
    val_losses.append(val_loss)

    print(f"Epoch {epoch+1}/{epochs}, Train Loss: {train_loss:.4f}, Val Loss: {val_loss:.4f}")

return train_losses, val_losses

```

```
train_losses, val_losses = train(model, train_loader, val_loader, criterion, optimizer, epochs=10)
```

```

plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Val Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.show()

```

```

def evaluate(model, test_loader):
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for images, labels in test_loader:
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    accuracy = correct / total
    print(f"Test Accuracy: {accuracy * 100:.2f}%")

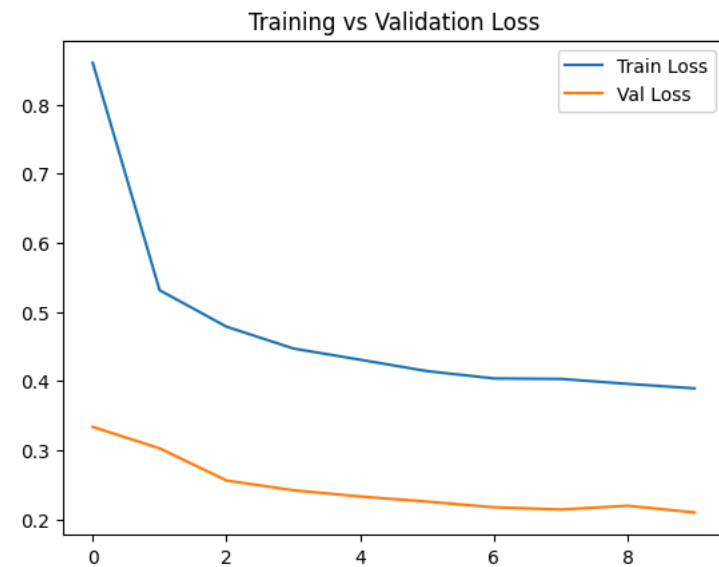
```

```
evaluate(model, test_loader)
```

```

Epoch 1/10, Train Loss: 0.8602, Val Loss: 0.3335
Epoch 2/10, Train Loss: 0.5313, Val Loss: 0.3025
Epoch 3/10, Train Loss: 0.4785, Val Loss: 0.2559
Epoch 4/10, Train Loss: 0.4469, Val Loss: 0.2418
Epoch 5/10, Train Loss: 0.4306, Val Loss: 0.2327
Epoch 6/10, Train Loss: 0.4142, Val Loss: 0.2254
Epoch 7/10, Train Loss: 0.4036, Val Loss: 0.2171
Epoch 8/10, Train Loss: 0.4029, Val Loss: 0.2139
Epoch 9/10, Train Loss: 0.3957, Val Loss: 0.2193
Epoch 10/10, Train Loss: 0.3892, Val Loss: 0.2097

```



Test Accuracy: 84.84%

New Section

